Algoritmos 2 Clase 6

Vínculos

- Clase: Algoritmos 2 Clase 6
- Definición de clases: Clases Grafos Algoritmos
- Ejercicios: Ejercicios Grafos

Estructuras Multienlazadas (Grafos)

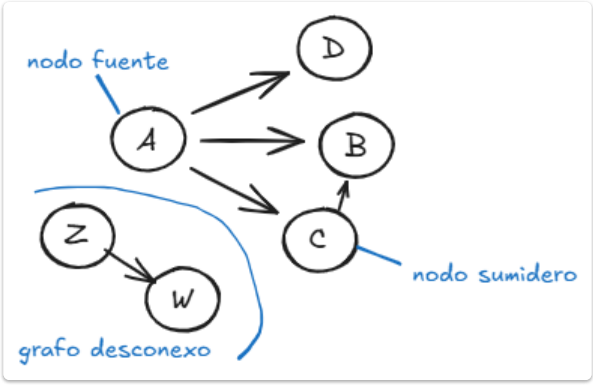
Un grafo es una estructura matemática del tipo

$$G = (V, A, \Phi)$$

Donde:

- V es el conjunto de Vértices
- A es el conjunto de Arcos
- Φ es una función de costos

$$\Phi : A(N, N) \to \mathbb{R}$$
$$V = [A, B, C]$$
$$A = [(A, B), (A, C), (A, D), (C, B)]$$



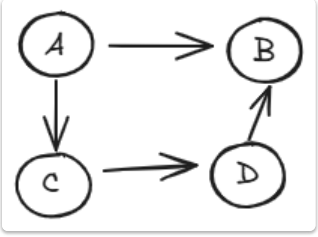
Conectividad

Conceptos clave para entender la relación entre los vértices:

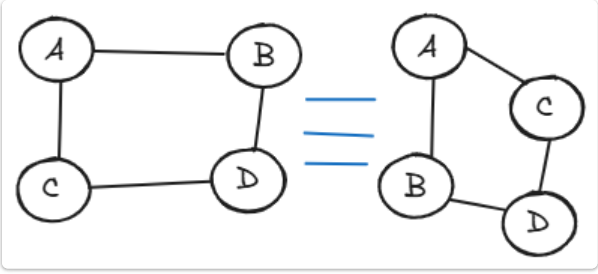
- **Grafo Conexo**: Existe al menos un camino entre cualquier par de vértices (en grafos no dirigidos).
- **Grafo Fuertemente Conexo**: En un grafo dirigido, existe un camino de u a v y de v a u para todo par de vértices.
- **Grado de un Vértice**:
  - **Grado de entrada (d_{in})**: Número de arcos que llegan al vértice.
  - **Grado de salida (d_{out})**: Número de arcos que salen del vértice.
  - En grafos no dirigidos, simplemente es el número de arcos incidentes.

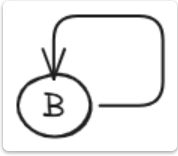
Tipos de Grafos

- **Dirigidos**: donde los arcos (las conexiones) tienen una dirección única.



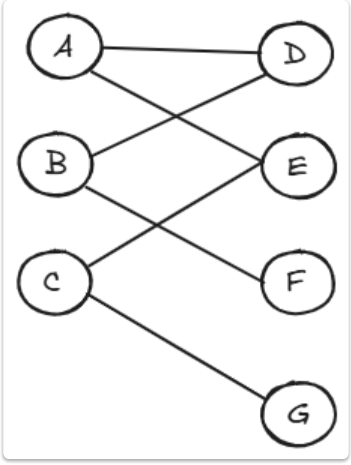
- **No Dirigidos**: donde los arcos actúan como autopistas de ida y vuelta, no tienen una dirección única, es bidireccional.



> *NOTA: trabajaremos con grafos simples, es decir, no se admiten los arcos (a,a)*
>
> 
>

Subtipos de grafos:

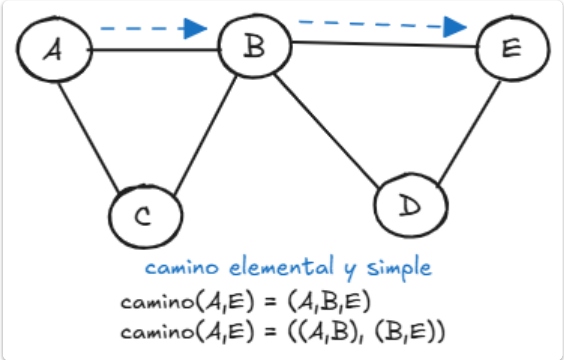
- **Planar**: se puede dibujar en $\mathbb{R}^2$ sin cruzar arcos.
- **Bipartitos**: se pueden obtener dos conjuntos de vértices A y B en los que los arcos no tocan elementos del mismo conjunto.
  - El conjunto de vértices se divide en dos subcojuntos V_1 y V_2.
  - Un grafo es bipartito si y solo si no posee ciclos de longitud impar.
  - Se puede colorear utilizando solo dos colores, de modo que ningún vértice adyacente tenga el mismo color.
  - Todos los árboles son grafos bipartitos



Caminos y circuitos

Caminos

- **Camino elemental**: no repite nodos
- **Camino Simple**: no repite arcos.

> *Todo camino elemental es simple pero no al revés.*
>
> 
>

Circuitos ó ciclos

Es un camino que parte de un nodo A y llega al mismo nodo A
$C(A, A) = (A, B, C, A)$

Caminos Especiales

- **Camino Euleriano**: camino que atraviesa todos los *arcos* una sola vez
- **Camino Hamiltoniano**: camino que atraviesa todos los *vértices* una sola vez

Algoritmos Clásicos (no evaluado en principio)

Si la función de costos es relevante, se utilizan algoritmos específicos:

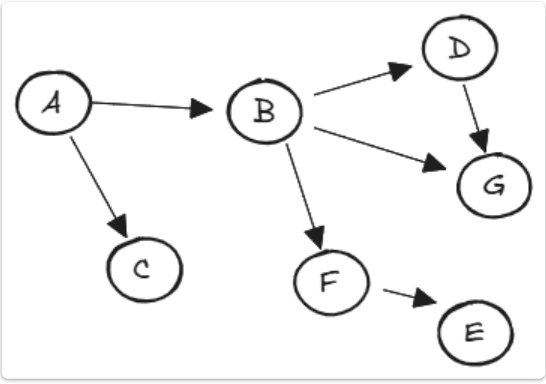
1. **Caminos Mínimos (Shortest Path)**:

- **Dijkstra**: Encuentra el camino más corto desde un origen a todos los demás (pesos no negativos).
- **Bellman-Ford**: Permite pesos negativos.

2. **Árbol de Recubrimiento Mínimo (MST)**:
   - **Kruskal**: Basado en arcos (usa *Union-Find*).
   - **Prim**: Basado en vértices (similar a Dijkstra).

Representaciones en Memoria

Matriz de Adyacencia:



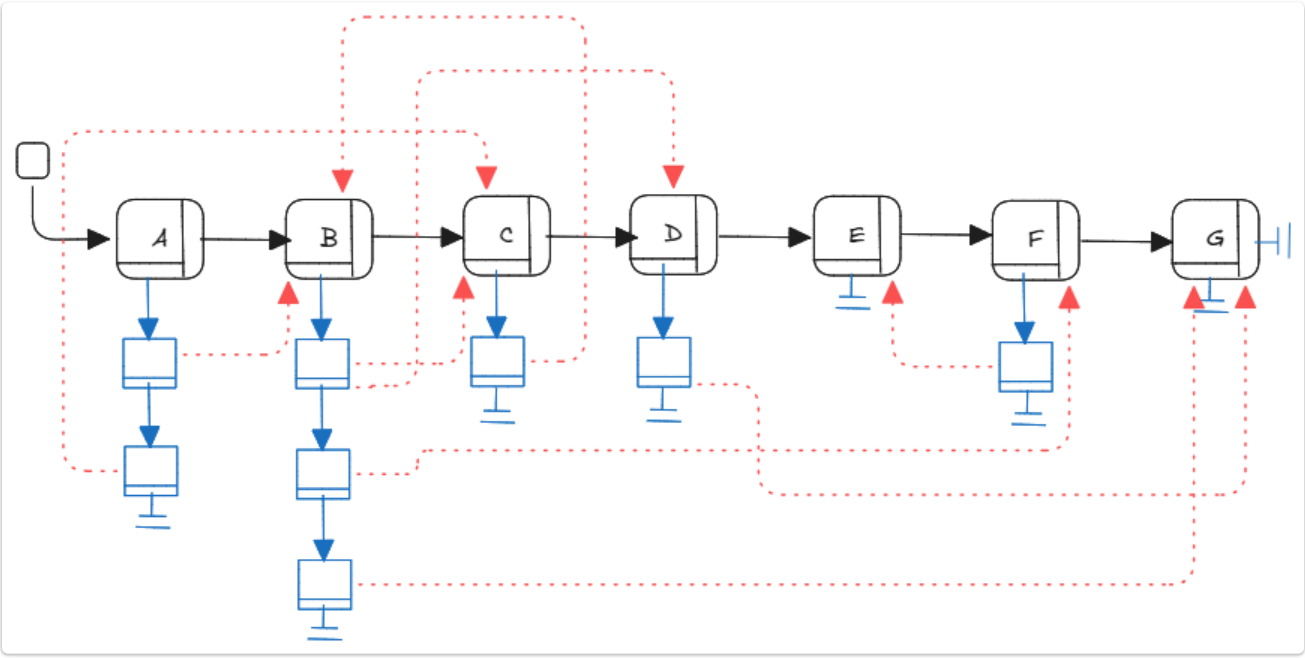
En memoria: $O(n^2)$; $n = |v|$

	A	B	C	D	E	F	G
A	0	1	0	0	0	0	0
B	0	0	0	1	0	1	1
C	0	1	0	0	0	0	0
D	0	0	0	0	0	0	1
E	0	0	0	0	0	0	0
F	0	0	0	0	1	0	0
G	0	0	0	0	0	0	0

Lista de adyacencia

Funciona con dos listas, una lista de los nodos vertices (nodos conectados linealmente) donde cada vértices tiene una lista de apuntadores a otros vértices (una lista de adyacencia).

- Es útil en grafos no densos
- **En memoria**: $O(n + m)$ donde $n = |V| \quad m = |A|$



Aplicaciones Prácticas

- **Redes de computadoras**: Enrutamiento de paquetes
- **Redes Sociales**: Sugerencias de amistad (amigos de amigos).
- **Mapas y GPS**: Cálculos de rutas óptimas (Google Maps).
- **Compiladores**: Análisis de dependencias entre módulos.

Definición de Clases

Clases Nodos

Nodo Arco

```
Class NodoArco
    Attributes
        Private:
            pointer to NodoVertice<Element>: info
            pointer to NodoArco<Element>: siguiente
            float: peso
    Methods
        Public:
            proc construir(...)
            // Getters
            ...
            // Setters
            ...
endClass
```

Nodo Vértice

```
Class NodoVertice
    Attributes
        Private:
            Element: info
            pointer to NodoVertice<Element>: siguiente
            pointer to NodoArco<Element>: listaAdy
    Methods
        Public:
            proc construir(Element: info, pointer to NodoVertice<Element>: sig, pointer to NodoArco<Element>:listaAdy)
            // Getters
            ...
            // Setters
            ...
endClass
```

Clase Grafo

> *En métodos como el DFS se utiliza un método interno privado recursivo de igual nombre con diferentes parámetros (polimorfismo).*
> *De manera similar, los métodos públicos recursivos requieren un método privado, ya sea polimórfico o no.*

```
Class Grafo
    Attributes
        Private:
            pointer to NodoVertice<Element>: g
            int: nVertices, nArcos
            bool: esDirigido
    Methods
        Public:
            proc constructor(...)
            // Getters
            ...
            // Setters
            ...

            func getVertices(): Lista<Element> // O(n)
```

```
            func getArcos(): Lista<Lista<Element>> // O(n + m)
            func getPesoArco(Element: v, w): float // O(n + arcos(v))
            func getSucesores(Element: v): Lista<Element> // O(n+ arcos(v)) Solo en dirigidos
            func getPredecesores(Element: v): Lista<Element> // O(n + m)
            func getGradoVertice(Element: v): int // O(n + arcos(v))
            func getVecinos(Element: v): Lista<Element> //O(n + arcos(v)) No dirigidos
            proc insertarVertice(Element:v) // O(1) Se inserta al principio
            proc insertarArco(Element: v, w, float: peso) // O(n)
            proc eliminarVertice(Element: v) // O(n + m)
            proc eliminarArco(Element: v, w) // O(n + arcos(v) + arcos(w))
            proc setPeso(Element: v, w, float: peso) // O(n + arcos(v) + arcos(w))
            func bfs(): Lista<Element> // O(n + m)
            func dfs(): Lista<Element> // O(n + m)
endClass
```